AWS Cloud Infrastructure Project

Complete Step-by-Step Technical Walkthrough & Reference Guide

Architecture: Custom VPC + EC2 + S3 + IAM Role + EBS Snapshot + CloudWatch + SNS + Bash Log Analytics

---

Phase 1: Isolated Network Architecture (Amazon VPC)

Step 1: Custom Virtual Private Cloud (VPC)

Tool / Service Used: Amazon VPC (Virtual Private Cloud Console)
How We Did It: Created a dedicated VPC named `InternshipAWS-VPC` with CIDR block `10.0.0.0/16`.
Technical Reason (Why it is used): Provides a logically isolated virtual network boundary in the AWS Cloud. Using a custom VPC instead of the shared default VPC is a strict production best practice that gives full control over IP address ranges, subnet segregation, routing tables, and perimeter security.
In Plain English (ELI5): We bought our own private, fenced-in plot of land inside the AWS cloud city. Nobody can enter or build on our land without our direct permission.

Step 2: Public Subnet Provisioning

Tool / Service Used: AWS VPC Subnets Console
How We Did It: Carved out a subnet named `InternshipAWS-PublicSubnet` with CIDR `10.0.1.0/24`, and toggled *'Enable auto-assign public IPv4 address'*.
Technical Reason (Why it is used): Subnets partition the VPC into distinct IP segments. Enabling the auto-assign public IPv4 flag ensures any web servers deployed here automatically get a routable public IP so outside users can access our website.
In Plain English (ELI5): We marked out a specific front yard on our land to build our house and made sure it has an official street address so delivery trucks can find it.

Step 3: Internet Gateway (IGW) Attachment

Tool / Service Used: Amazon VPC Internet Gateway Console
How We Did It: Created an Internet Gateway named `InternshipAWS-IGW` and attached it to `InternshipAWS-VPC`.
Technical Reason (Why it is used): By default, an AWS VPC is completely isolated from the internet. The Internet Gateway acts as a horizontally scaled, redundant software router that performs network address translation (NAT) between the VPC and the public internet.
In Plain English (ELI5): We put a large front gate in our fence so people from the outside world can walk in and out.

Step 4: Route Table Association & Outbound Route

Tool / Service Used: AWS VPC Route Tables Console
How We Did It: Associated our public subnet with a custom Route Table, and added a default route with destination `0.0.0.0/0` pointing to `InternshipAWS-IGW`.
Technical Reason (Why it is used): A subnet is not truly public until its routing table directs external traffic (`0.0.0.0/0` represents all internet addresses) out through the Internet Gateway.
In Plain English (ELI5): We put up road signs inside our property that say: *'If you want to go anywhere in the outside world, follow this path out the front gate.'*

Phase 2: Compute Deployment & Security Hardening

Step 5: Security Group Configuration (Virtual Firewall)

Tool / Service Used: AWS EC2 Security Groups (Stateful Firewall)
How We Did It: Created security group `InternshipAWS-SG` with two specific inbound rules:
- **SSH (Port 22):** Restricted exclusively to `My IP` (admin personal IP /32).
- **HTTP (Port 80):** Open to `0.0.0.0/0` (Anywhere).
Technical Reason (Why it is used): Implements the *Principle of Least Privilege*. Leaving SSH open to the whole internet invites constant brute-force attacks and vulnerability bots. Locking SSH to our exact IP protects server management, while keeping HTTP open allows public website visitors.
In Plain English (ELI5): We hired a security bouncer for our house. He lets any customer look through the restaurant window (HTTP), but only lets you enter through the private back office door using your key (SSH).

Step 6: EC2 Virtual Server Provisioning

Tool / Service Used: Amazon EC2 (Elastic Compute Cloud Console)
How We Did It: Launched a `t3.micro` instance running Ubuntu Linux in `InternshipAWS-PublicSubnet`, attaching our key pair (`.pem`) and security group.
Technical Reason (Why it is used): EC2 provides scalable on-demand cloud computing. Selecting `t3.micro` (2 vCPUs, 1 GiB memory) complies with the AWS Free Tier (750 free hours/month) while providing burstable compute capability.
In Plain English (ELI5): We set up our virtual computer in the house to run our online business.

Step 7: Web Server Setup & Custom Endpoints

Tool / Service Used: SSH Client, Linux package manager (`apt`), Apache2 HTTP Server
How We Did It: Connected via SSH (`ssh -i key.pem ubuntu@IP`), ran `sudo apt update && sudo apt install apache2 -y`, and created custom web files at `/var/www/html/index.html` and `/var/www/html/health`.
Technical Reason (Why it is used): Apache is a dependable web server daemon listening on port 80. The `/health` endpoint is an industry standard used by monitoring systems and load balancers to quickly verify if the server is healthy without loading heavy website assets.
In Plain English (ELI5): We hired a digital waiter who immediately hands customers our website menu whenever they knock on the front door.

Phase 3: Identity & Keyless Storage Architecture

Step 8: IAM Instance Role (Zero Hardcoded Credentials)

Tool / Service Used: AWS IAM (Identity and Access Management)
How We Did It: Created role `InternshipAWS-EC2-Role` with trusted entity `ec2.amazonaws.com`. Attached a least-privilege inline policy allowing `s3:ListBucket`, `s3:GetObject`, and `s3:PutObject` specifically on our bucket, then attached it to the EC2 instance.
Technical Reason (Why it is used): Storing static AWS access keys on a server is an extreme security risk (if the server is compromised, credentials are stolen). An IAM Instance Role provides temporary, automatically-rotated credentials delivered through the AWS Instance Metadata Service (IMDS).
In Plain English (ELI5): Instead of taping our master account password to the computer desk, we gave the computer a temporary security badge that only lets it drop files into our storage locker.

Step 9: S3 Bucket Creation & Automated Log Archival

Tool / Service Used: Amazon S3 & AWS CLI v2
How We Did It: Created private S3 bucket `internshipaws-logs-karthik` with Block Public Access enabled. Installed AWS CLI v2 and ran `aws s3 sync /var/log/apache2/ s3://internshipaws-logs-karthik/logs/$(date +%F)/`.
Technical Reason (Why it is used): If a server runs out of disk space or crashes, locally stored logs are lost forever. Offloading logs to Amazon S3 guarantees 99.999999999% (11 9s) durability, centralized compliance archiving, and cost-effective long-term storage.

In Plain English (ELI5): We rented a bank vault down the road and send a copy of our visitor diary there every day for safekeeping.

Step 10: EBS Volume Disaster Recovery Snapshot

Tool / Service Used: Amazon EBS (Elastic Block Store) Console
How We Did It: Selected the EC2 instance's 8 GB root volume, clicked *'Create Snapshot'*, tagged it `InternshipAWS-Backup`, and confirmed the status changed to *'Completed'*.
Technical Reason (Why it is used): Provides point-in-time, incremental block storage backup. If the OS gets corrupted or an update fails, a brand new EBS volume or EC2 instance can be spawned directly from this snapshot in minutes with zero loss.
In Plain English (ELI5): We took a complete photo copy of the computer's hard drive so that if it ever breaks, we can rebuild the exact same computer immediately.

Phase 4: Proactive Observability & Cost Control

Step 11: Alerting Pipeline Setup (Amazon SNS)

Tool / Service Used: Amazon SNS (Simple Notification Service)
How We Did It: Created an SNS Topic named `InternshipAWS-Alerts`, subscribed an email address, and clicked the verification link in the received email to confirm the subscription.
Technical Reason (Why it is used): Decouples system metrics from notification endpoints. SNS handles fan-out message delivery to emails, SMS, or Slack webhooks whenever connected services trigger an event.
In Plain English (ELI5): We hired a messenger pigeon whose only job is to fly directly to our email inbox whenever something goes wrong.

Step 12: High-CPU CloudWatch Alarm

Tool / Service Used: Amazon CloudWatch Alarms
How We Did It: Created an alarm tracking metric `CPUUtilization` with condition `>= 70%` for 1 consecutive 5-minute period, configured to publish an alert to our SNS Topic.
Technical Reason (Why it is used): Automates system observability. Rather than manually monitoring server dashboards, CloudWatch watches CPU telemetry 24/7 and automatically triggers an incident response if the server is overloaded.
In Plain English (ELI5): We attached a heart-rate monitor to our computer that sounds an alarm if the computer's heart beats too fast for 5 minutes.

Step 13: CPU Stress Testing & Alarm Validation

Tool / Service Used: Linux `stress` benchmark tool
How We Did It: Installed `stress` on Ubuntu and ran `stress --cpu 2 --timeout 400` across both vCPUs of our t3.micro for ~6 minutes until the alarm transitioned to *'In alarm'* and the alert email arrived in our inbox.
Technical Reason (Why it is used): Crucial testing step: A t3.micro has 2 vCPUs. Running stress on only 1 vCPU caps overall CPU at 50%, which fails to cross a 70% alarm threshold. Stressing 2 vCPUs drove CPU to 100%, proving the entire detection and notification pipeline functions in production.
In Plain English (ELI5): We ran a simulated fire drill to make sure our fire alarm and emergency phone calls actually work when real heat hits the room.

Step 14: AWS Zero-Spend Budget

Tool / Service Used: AWS Billing & Budgets Console
How We Did It: Configured a Zero-Spend Budget with a threshold of $0.01 and alert recipient email.
Technical Reason (Why it is used): Financial guardrail. It guarantees you are alerted the moment unexpected costs or non-Free Tier services are accidentally turned on.
In Plain English (ELI5): We put an alarm on our wallet that beeps loudly if AWS tries to charge us

even one single cent.

Phase 5: Log Analytics & Version Control

Step 15: Traffic Simulation & Bash Log Analysis Script

Tool / Service Used: Linux Bash, `curl`, `awk`, `grep`, `sort`, `uniq`
How We Did It: Generated 400+ simulated requests (300 successful requests and 100 requests to non-existent URLs to generate 404 errors). Created and executed `analyze_logs.sh`:
- Top Requested Endpoints: `awk '{print $7}' /var/log/apache2/access.log | sort | uniq -c | sort -rn | head -n 3`
- HTTP 4xx/5xx Error Count: `awk '{print $9}' /var/log/apache2/access.log | grep -E "^[45]" | wc -l`
- Unique Visitor Count: `awk '{print $1}' /var/log/apache2/access.log | sort | uniq | wc -l`
Technical Reason (Why it is used): Demonstrates native Linux systems engineering. Engineers must be able to parse log files and diagnose incident metrics without relying on expensive SaaS tools.
In Plain English (ELI5): We built a mini speed-reading robot that read through hundreds of visitor entries in 1 second and immediately summarized the errors and top pages.

Step 16: Git Version Control & Public GitHub Portfolio

Tool / Service Used: Git CLI & GitHub
How We Did It: Initialized local Git repository, created an employer-facing `README.md` and documentation, committed all artifacts, and pushed to `https://github.com/karthik062004/AWS-Cloud-Project`.
Technical Reason (Why it is used): Professional proof of work. Version-controlling infrastructure code and scripts provides hiring managers and technical recruiters with transparent evidence of hands-on technical proficiency.
In Plain English (ELI5): We framed our completed project neatly and hung it in our public digital gallery for the world to see.